

# DS 705 Final Project

Jim Ryan

03/01/2020

## Part 2: Introduction

For this project I will use logistic regression to predict which applicants are likely to default on their loans. I am going to use the loans50k data set that was given to us as part of this assignment. This data set contains 50,000 loans in various statuses and various amounts. There are 32 available variables in the data set. My response variable will be the status variable. I will start by preparing and cleaning the data.

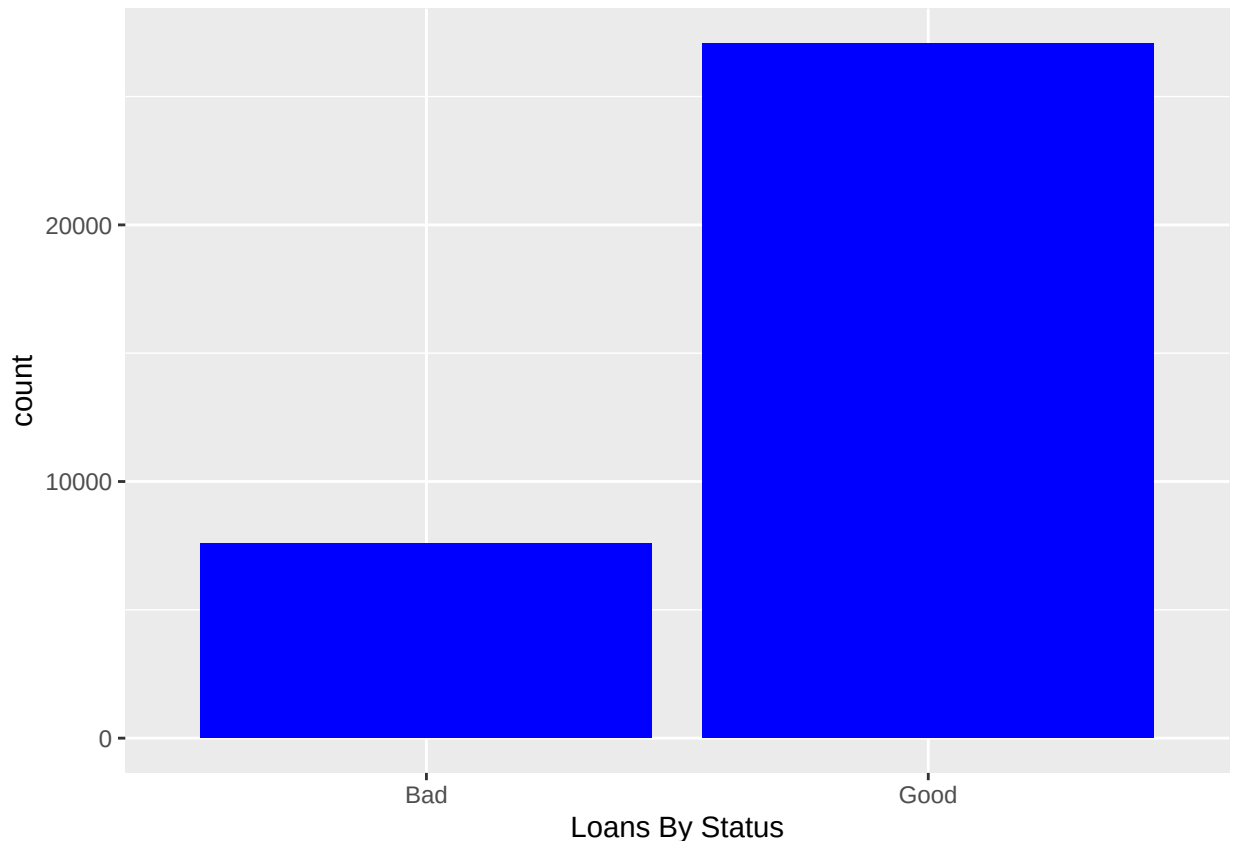
```
LoansDT <- fread("Loans50k.csv")
```

## Part 3: Preparing and Cleaning the Data

The data contains different variables collected as either part of the application process or information from when the loan was disbursed. I am trying to predict which variable or variables will allow me to predict whether a loan becomes default or not. I removed all loans that did not have a status of 'Fully Paid', 'Default' or 'Charged Off'. Then I copied the response variable – status – into another field called status2. I then updated the status of loans that were Fully Paid to 'Good' and the ones that were 'Default' or 'Charged Off' to 'Bad'.

There are 27,074 loans or 78% with a status of 'Good' and 7,581 loans or 22 percent with a status of 'Bad'. Here is a Histogram of that shows the number of good and bad loans after the status has been updated.

```
#bar plot on status
p1 <- ggplot(data=LoansDT, aes(x=status2)) +
  geom_bar(fill="blue") +
  xlab("Loans By Status")
p1
```



```
#Update good to 1 bad to 0 and make var a factor
LoansDT <- sqldf(c("UPDATE LoansDT SET status2 = 1 where status2 = 'Good'", "UPDATE LoansDT SET status2 = 0 where status2 = 'Bad'"))
```

```
## Warning in result_fetch(res@ptr, n = n): SQL statements must be issued with
## dbExecute() or dbSendStatement() instead of dbGetQuery() or dbSendQuery().
```

```
## Warning in result_fetch(res@ptr, n = n): SQL statements must be issued with
## dbExecute() or dbSendStatement() instead of dbGetQuery() or dbSendQuery().
```

```
# set status field as factor
LoansDT$status2 <- as.factor(LoansDT$status2)
```

I used `summary()` with a `sum` on the `NA` in the data set to determine which variable had `NA` in them and how many there were. I found that the only 3 variables that had `NA` values in them were `revolRatio` (15 `NA`), `bcOpen` (360 `NA`) and `bcRatio` (384 `NA`). Since the missing values were only around 1 percent of the total number for these variables, I imputed them using the mean of all the values for that specific variable.

```
##      loanID      amount      term      rate      payment      grade
##           0           0           0           0           0           0
```

```
## employment      length      home      income      verified      status
##           0           0           0           0           0           0
##      reason      state      debtIncRat      delinq2yr      inq6mth      openAcc
##           0           0           0           0           0           0
##      pubRec      revolRatio      totalAcc      totalPaid      totalBal      totalRevLim
##           0           15           0           0           0           0
##      accOpen24      avgBal      bcOpen      bcRatio      totalLim      totalRevBal
##           0           0           360           384           0           0
##      totalBcLim      totalIllLim      status2      amount2
##           0           0           0           0
```

Then I focused my attention on the employment variable. There were 15,268 different values for employment including 1918 where the value is blank. I selected all rows where the value had a count greater than 100. I updated 1789 values of the employment variable that had the word manager in the to be just 'Manager'. There were 934 different variations of the word 'Teacher', so I updated them all to be teacher.

The 'Grade variable was updated from 'A' thru 'G' to 0 thru 6 and made into a factor. I update the 'verified' variable by updating those with a value of 'Source Verified' to 'Verified' and made that variable a factor. I did something similar with the home variable. I set the variable to OWN when the value was mortgage so then there were just 2 categorical variables - 'own' and 'rent' made the variable a factor. The variable term only had 2 values (36 months and 60 months) so the variable term2 was updated to a factor.

```
sqldf("Select count(*) from LoansDT where employment like '%manager%' ")
```

```
##      count(*)
## 1          5228
```

```
# update all employment rows with word manager in the to Manager
```

```
LoansDT <- sqldf(c("update LoansDT set employment = 'manager' where employment like '%m
```

```
# update all employment rows with word manager in the to manager
```

```
LoansDT <- sqldf(c("update LoansDT set employment = 'Teacher' where employment like '%t
```

```
#Update grade to numeric (0 thru 6)
```

```
LoansDT <- sqldf(c("UPDATE LoansDT SET grade = CASE grade WHEN 'A' THEN 0 WHEN 'B' THEN
```

```
# set grade as a factor
```

```
LoansDT$grade <- as.factor(LoansDT$grade)
```

```
#Update home to OWN when value is mortgage, set own = 1 and rent = 0 and make it a fa
```

```
LoansDT <- sqldf(c("Update LoansDT set home = 'OWN' where home = 'MORTGAGE'", "Select *
```

```
LoansDT$home <- as.factor(LoansDT$home)
```

```

# Update Source Verified and Verified to 1 Not verified as 0 make it a factor
LoansDT <- sqldf(c("UPDATE LoansDT SET verified = 'Verified' where verified = 'Source Verified'"))

LoansDT$verified <- as.factor(LoansDT$verified)

# make term column a factor
LoansDT$term <- as.factor(LoansDT$term)

```

I removed the following variables because more than 25 percent of them were 0: pubRec, delinq2yr, and inq6mth. I also dropped the original status variable.

```

# remove original status and original term columns
LoansDT$status <- NULL

colnames(LoansDT)

```

```

## [1] "loanID"      "amount"      "term"        "rate"        "payment"
## [6] "grade"       "employment"  "length"      "home"        "income"
## [11] "verified"    "reason"      "state"       "debtIncRat"  "delinq2yr"
## [16] "inq6mth"     "openAcc"     "pubRec"      "revolRatio"  "totalAcc"
## [21] "totalPaid"   "totalBal"    "totalRevLim" "accOpen24"   "avgBal"
## [26] "bcOpen"      "bcRatio"     "totalLim"    "totalRevBal" "totalBcLim"
## [31] "totalIllLim" "status2"     "amount2"

```

```
sqldf ("select count (*) from LoansDT where pubRec = 0 ")
```

```

## count (*)
## 1      28076

```

```
sqldf ("select count (*) from LoansDT where delinq2yr = 0 ")
```

```

## count (*)
## 1      27600

```

```
sqldf ("select count (*) from LoansDT where inq6mth = 0 ")
```

```

## count (*)
## 1      19237

```

```
sqldf ("select count(distinct(employment)) from LoansDT")
```

```
## count(distinct(employment))  
## 1 13368
```

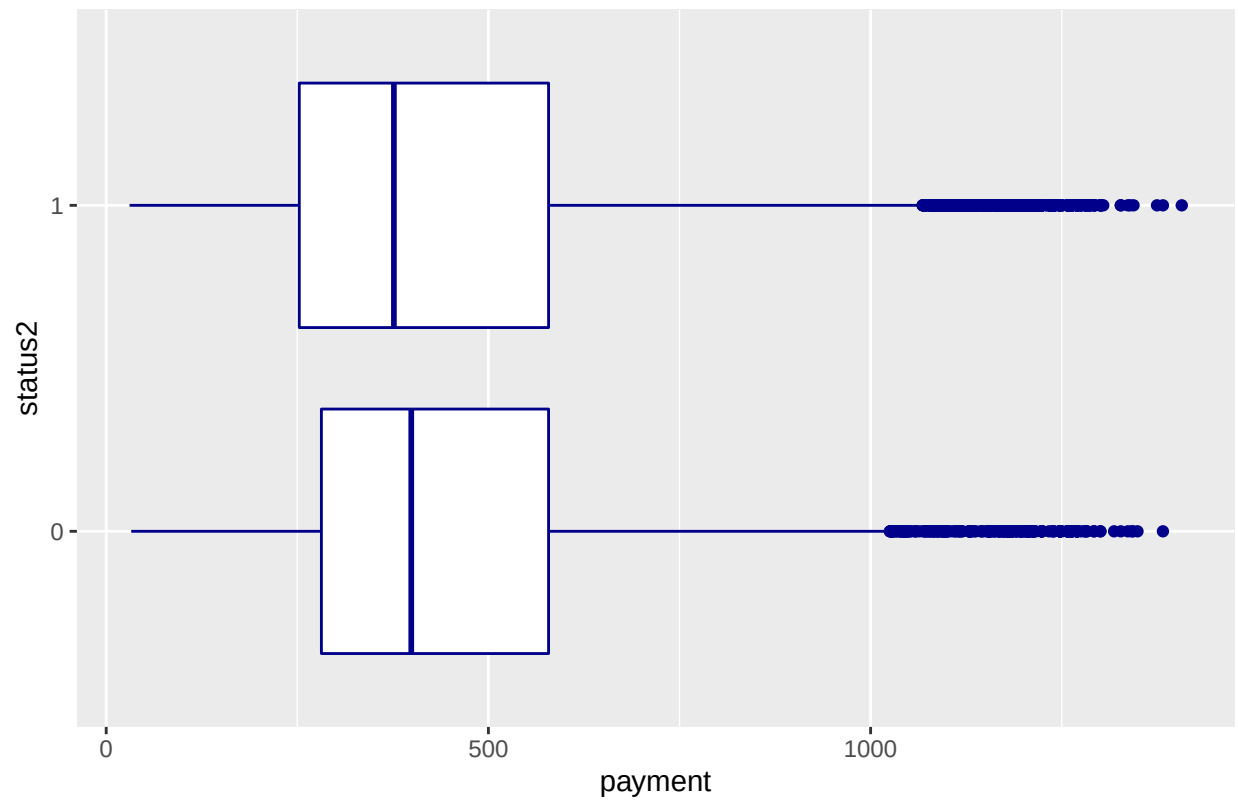
```
# following columns are removed as over 25% of values are blank or 0  
LoansDT$pubRec <- NULL  
LoansDT$delinq2yr <- NULL  
LoansDT$inq6mth <- NULL
```

#### Part 4: Exploring and Transforming the Data

Now I delve into some data exploration and transformation. First I do some exploration on the data set as a whole. I create box plots of payment and status to see if there is an observable relationship between payment and status. There is no observable relationship between payment and status for the entire dataset. Then I did the same for openAcc income and loanID. The only one that showed an obvious relationship was totalBal. The 'LoanID' variable looks constant no matter what the status. I assume this is just a loan Identifier and will drop it. The other 3 variables I will keep for part 5.

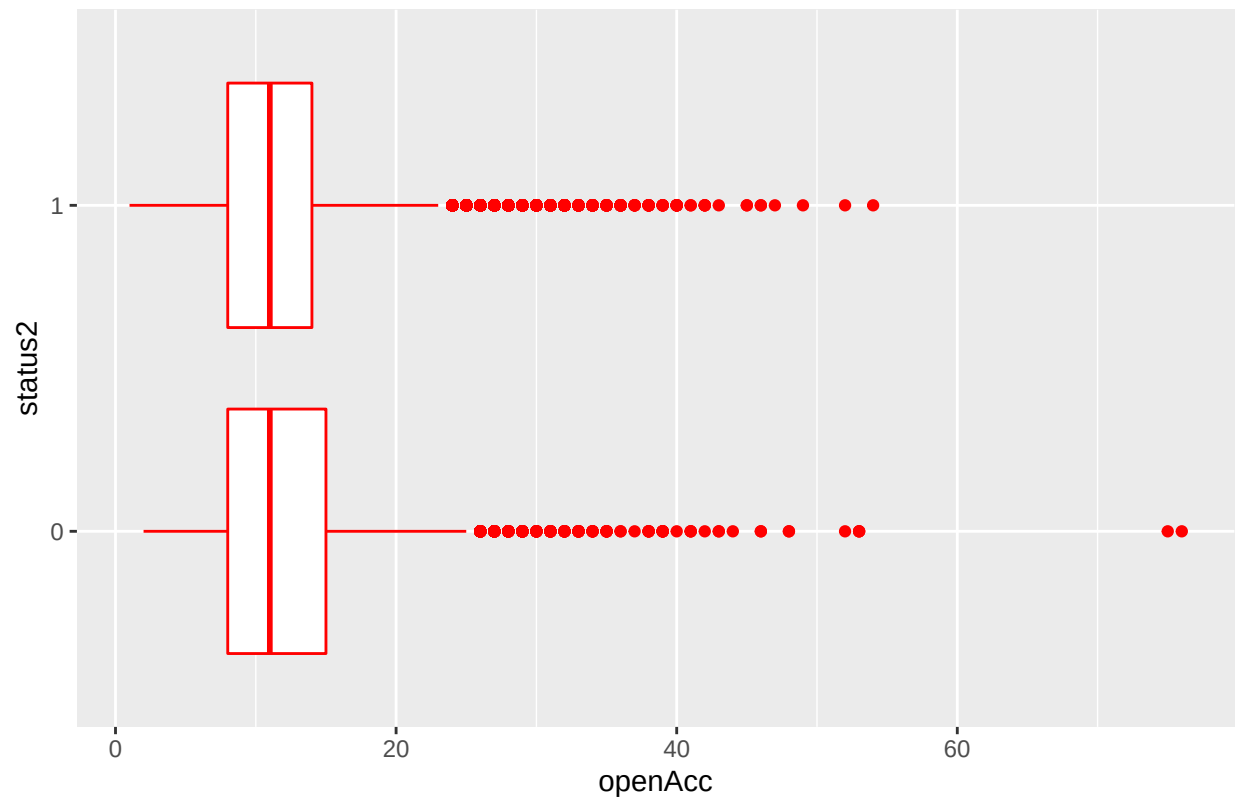
```
par(mfrow=c(2,2))  
ggplot(aes(x=payment, y=status2), data=LoansDT) +  
  geom_boxplot(color='darkblue') + ggtitle("Plot of status by Payment")
```

Plot of status by Payment



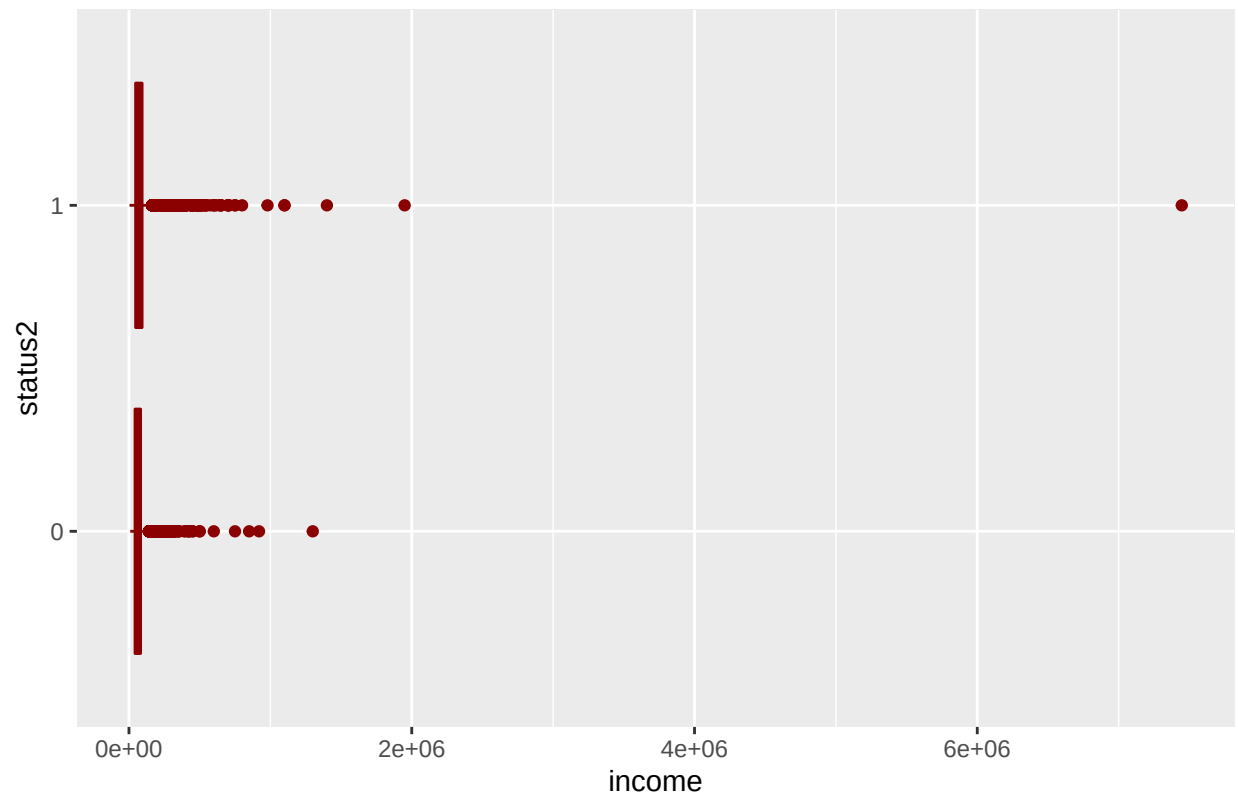
```
ggplot(aes(x=openAcc,y=status2),data=LoansDT)+  
  geom_boxplot(color='red') + ggtitle("Plot of status by openAcc")
```

Plot of status by openAcc

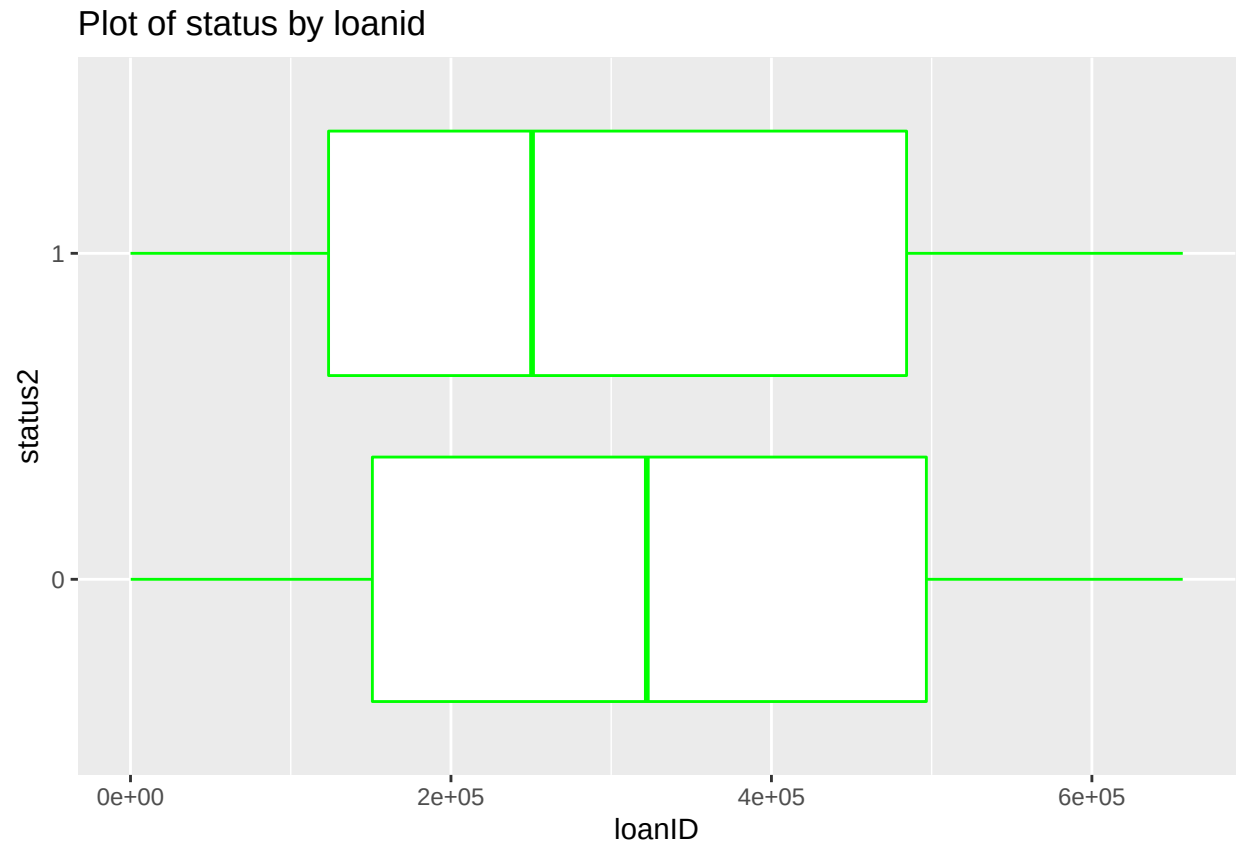


```
ggplot(aes(x=income,y=status2),data=LoansDT)+  
  geom_boxplot(color='darkred') + ggtitle("Plot of status by Income")
```

Plot of status by Income



```
ggplot(aes(x=loanID,y=status2),data=LoansDT)+  
  geom_boxplot(color='green') + ggtitle("Plot of status by loanid")
```



```
LoansDT$loanID <- NULL
```

Then I divided up my modified data frame into good and bad loans based on the status2 variable. Then I made bar graphs of some of the categorical variables divided by good and bad loans to see if there were any obvious relationships. I plotted the 'term', 'verified' and home variable against status2. For the term variable 'bad' loans have a greater percentage of loans with 60 month terms than the 'good' loans. It appears that there might be some relationship between the 'verified' and term variables and status. I will keep these variables for part 5.

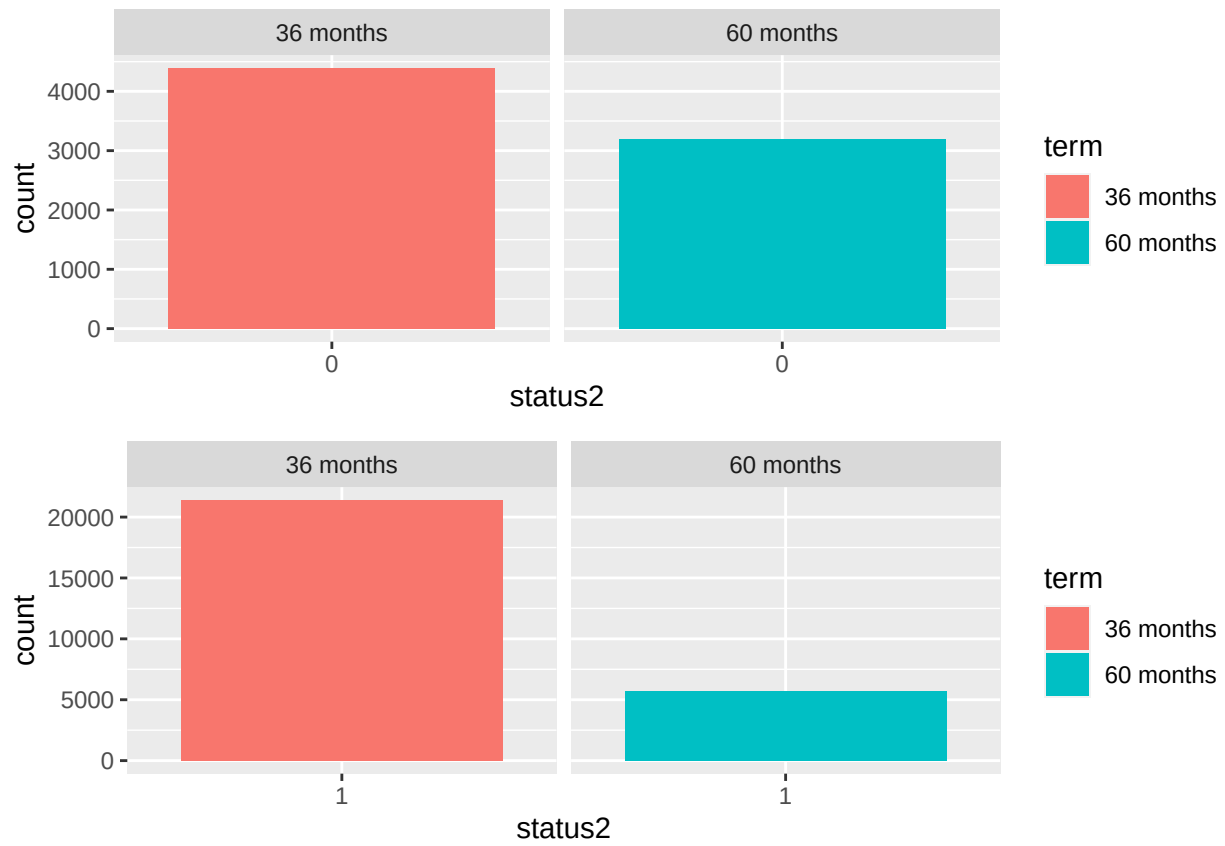
```
library("gridExtra")
loans_bad <- sqldf("Select * from LoansDT where status2 = 0")
loans_good <- sqldf("Select * from LoansDT where status2 = 1")

p1 <- ggplot(loans_bad, aes(status2, fill = term)) + facet_wrap(~term) + geom_bar()
p2 <- ggplot(loans_good, aes(status2, fill = term)) + facet_wrap(~term) + geom_bar()

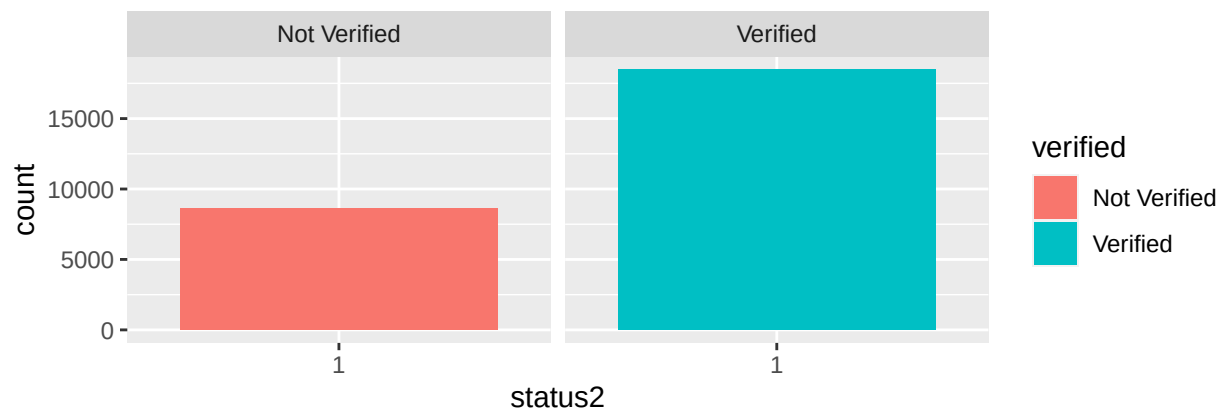
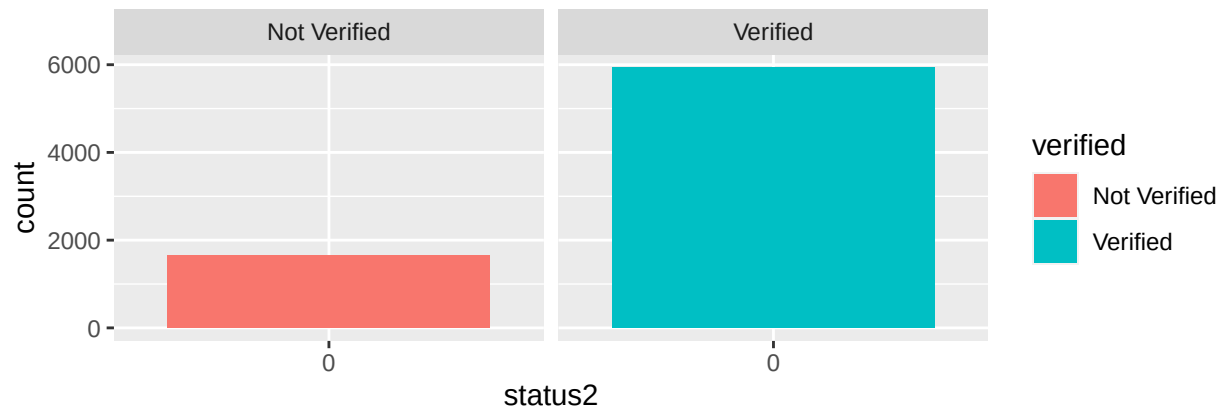
p3 <- ggplot(loans_bad, aes(status2, fill = verified)) + facet_wrap(~verified) + geom_bar()
p4 <- ggplot(loans_good, aes(status2, fill = verified)) + facet_wrap(~verified) + geom_bar()
```

```
p5 <- ggplot(loans_bad, aes(status2, fill = home)) + facet_wrap(~home) + geom_bar()
p6 <- ggplot(loans_good, aes(status2, fill = home)) + facet_wrap(~home) + geom_bar()

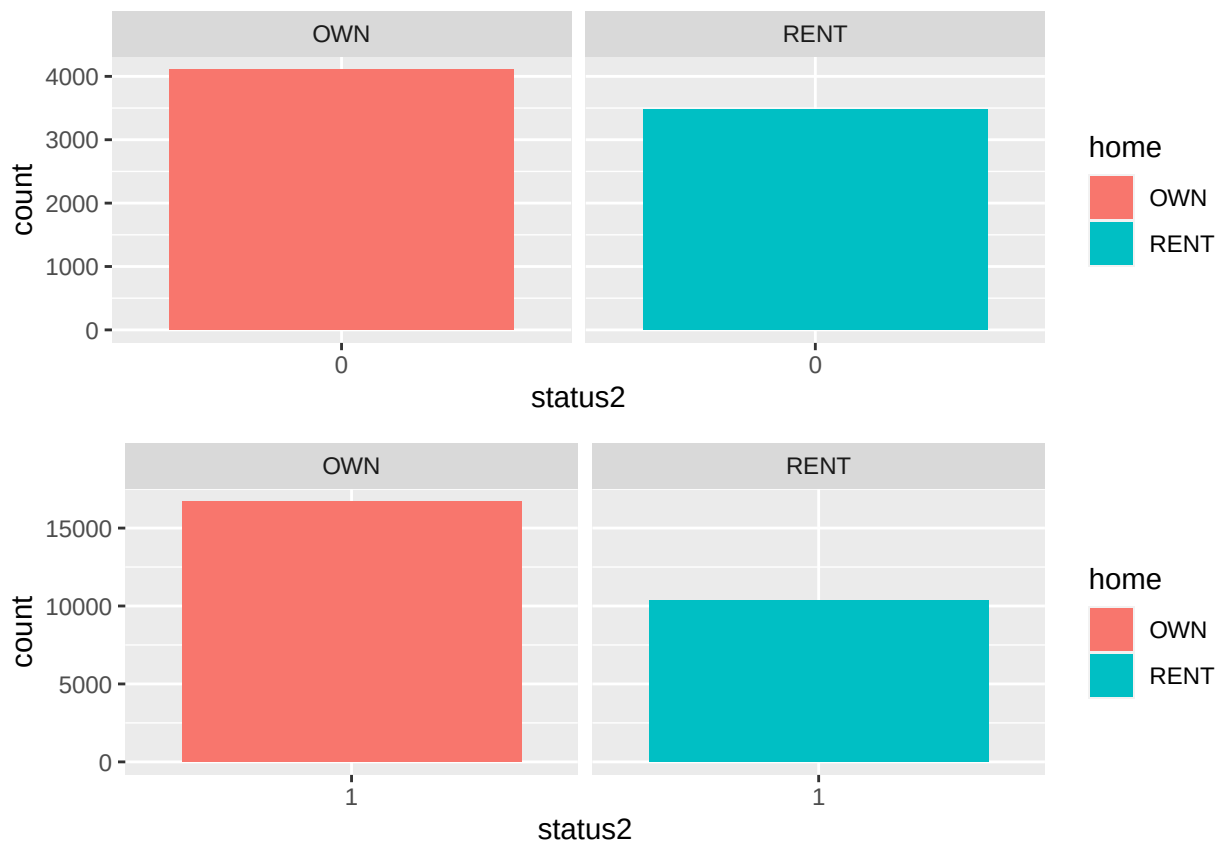
grid.arrange(p1,p2)
```



```
grid.arrange(p3,p4)
```



```
grid.arrange(p5,p6)
```



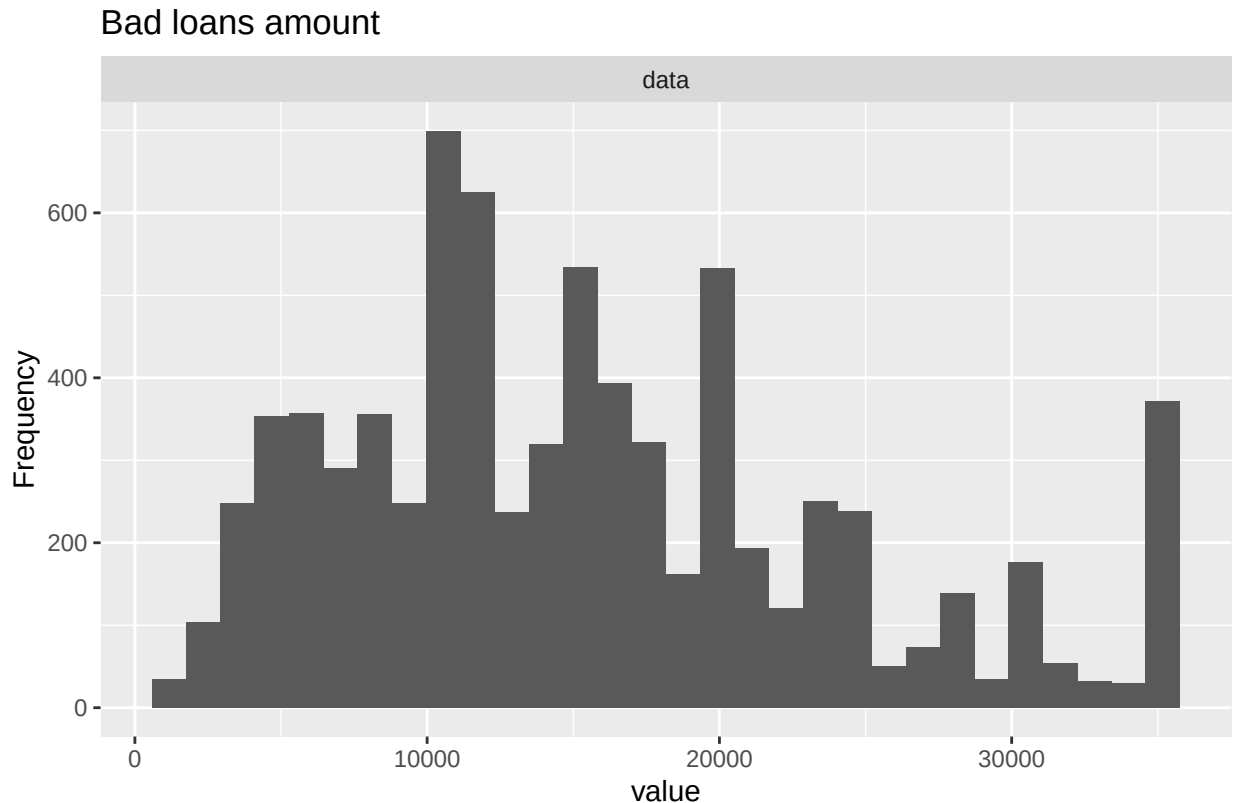
I plotted a histogram of all the variables in the LoansDT dataframe to get an overall sense of if the data is normally distributed or not. Due to space limitations, I did not include this plot. Much of the data looks right skewed.

I plotted the states against status for both the good and bad loans but noticed any relationship. As expected the more populous states (New York and California) had the higher number of loans both good and bad. I will be dropping this variable. I am also going to drop the employment field as there are so many different values in there it will slow my regression down as there are over 15,000 distinct values, which is too many for the variable to be significant.

I plotted variables from each dataframe ('Good' and 'Bad') to determine if the data is normally distributed or not. There is a definite right skew to many of the numeric variables. "rate", "Amount", "debt to income ratio" look normally distributed so I will not transform them. To transform the rest of the numeric variables, I created 2 new dataframes for the good and bad loans and with just the numeric fields I am going to transform. Then I used the log1p function on the dataframes. I use this function instead of the log function because some of the variables contain zeroes and log has issues with zeroes. I will not transform the following variables as they look normal: "revolRatio", "accOpen24" and "accOpen24". I plot the data into a histogram again after the log and the data looks much more normally distributed. There are a few that now look left-skewed. Also after performing it the first time and looking for NA's I found that it created a total of 384 NAs in bcratio between the 2 groups so I am removing that from the dataframes I apply the log1p to. The debtIncRat and

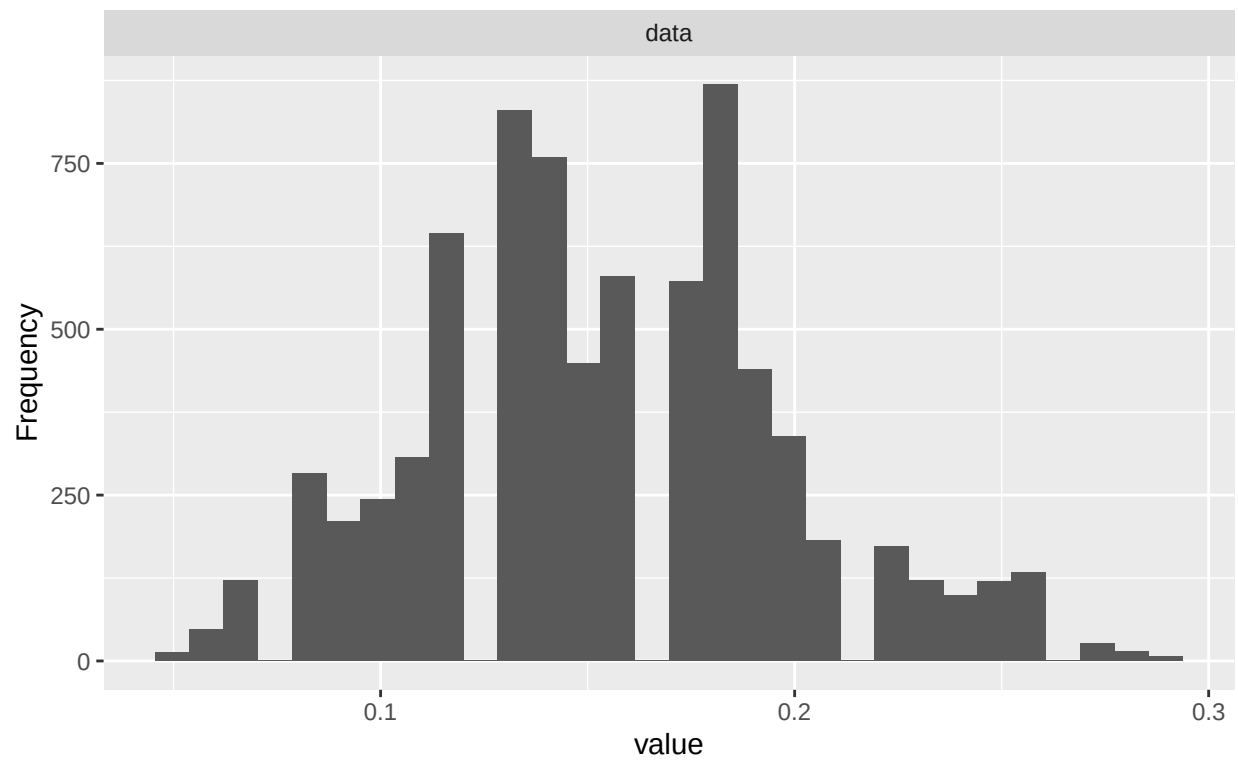
rate fields were also normally distributed so I did not include them in the group of variables to apply the log1p to. I also did not transform “totalPaid” as we are not supposed to use that values as a predictor. I split the categorical variables into a separate dataframe to join with the logged dataframes later. I plotted several of the variables after the log was applied to the to check the results.

```
par(mfrow=c(4,2))
plot_histogram(loans_bad$amount,title="Bad loans amount")
```



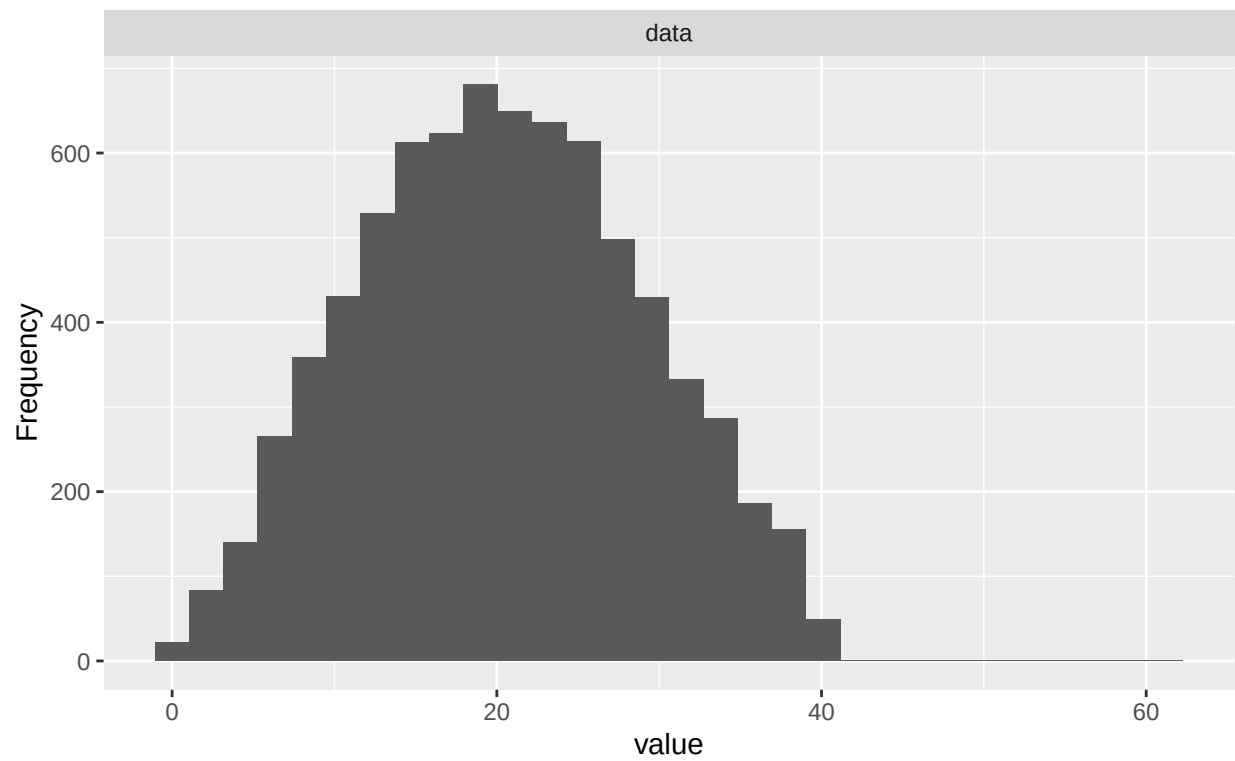
```
plot_histogram(loans_bad$rate,title="Bad loans rate")
```

Bad loans rate



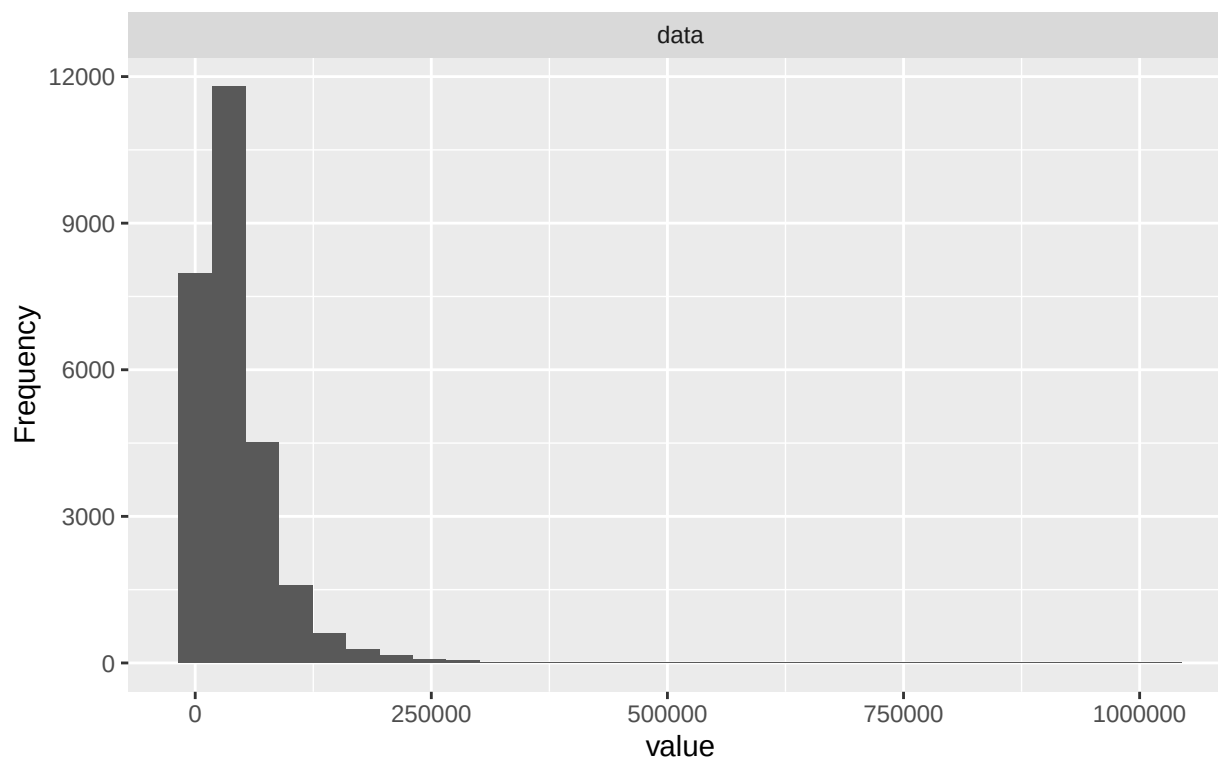
```
plot_histogram(loans_bad$debtIncRat,title="bad loans Debt to income ratio")
```

bad loans Debt to income ratio



```
plot_histogram(loans_good$totalIllim,title="Good loans TotalIllim")
```

Good loans TotalIlim



```
nologvars <- subset(LoansDT,select=c(bcRatio,debtIncRat,rate,totalPaid,amount2))

good_loan_numeric <- loans_good [ c("amount" , "payment" , "income" , "openAcc" , "re
                                "totalBal"      , "totalRevLim" , "accOpen24" , "avg
                                "totalRevBal" , "totalBcLim" , "totalIilLim")]

bad_loan_numeric <- loans_bad[ c("amount" , "payment" , "income" , "openAcc"      , "re
                                "totalBal"      , "totalRevLim" , "accOpen24" , "avg
                                "totalRevBal" , "totalBcLim" , "totalIilLim")]

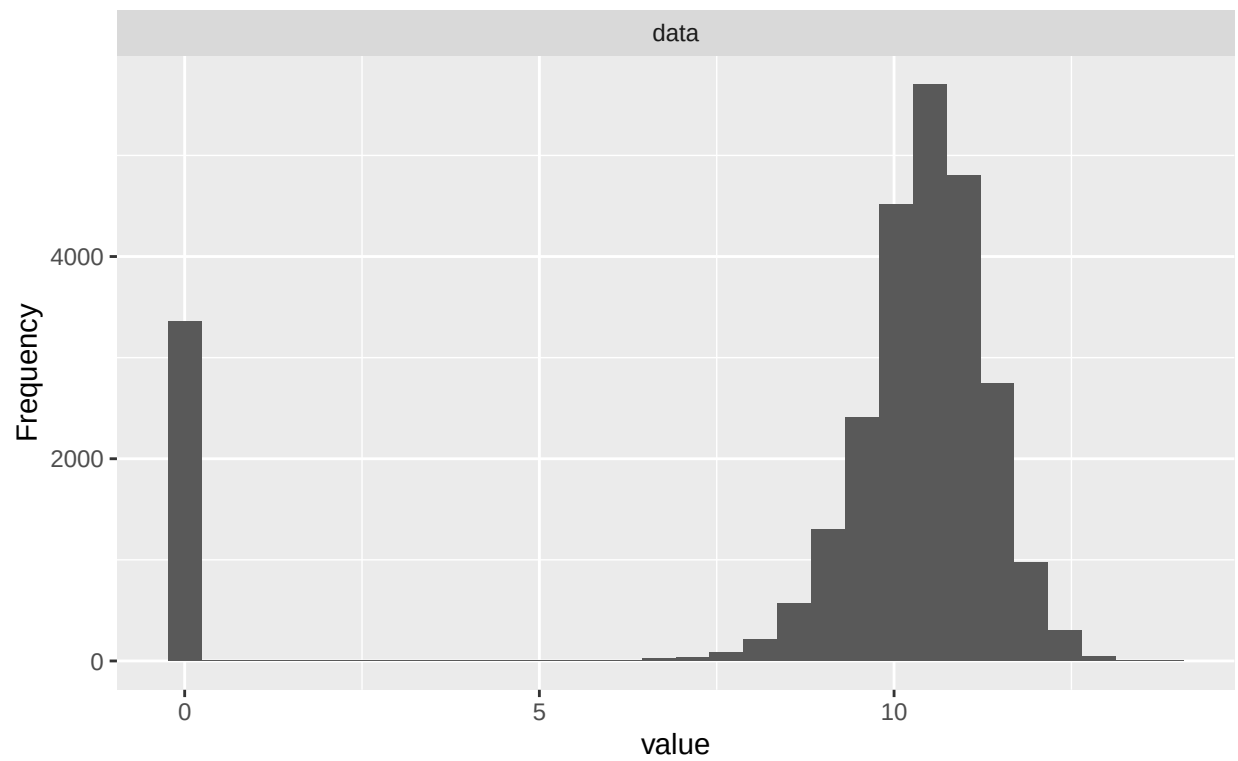
good_loan_discrete <- loans_good[ c("grade", "length" ,"home" ,"verified","reason" ,
                                "status2","term" )]

bad_loan_discrete <- loans_bad[ c("grade","length" ,"home" ,"verified","reason",
                                "status2","term")]

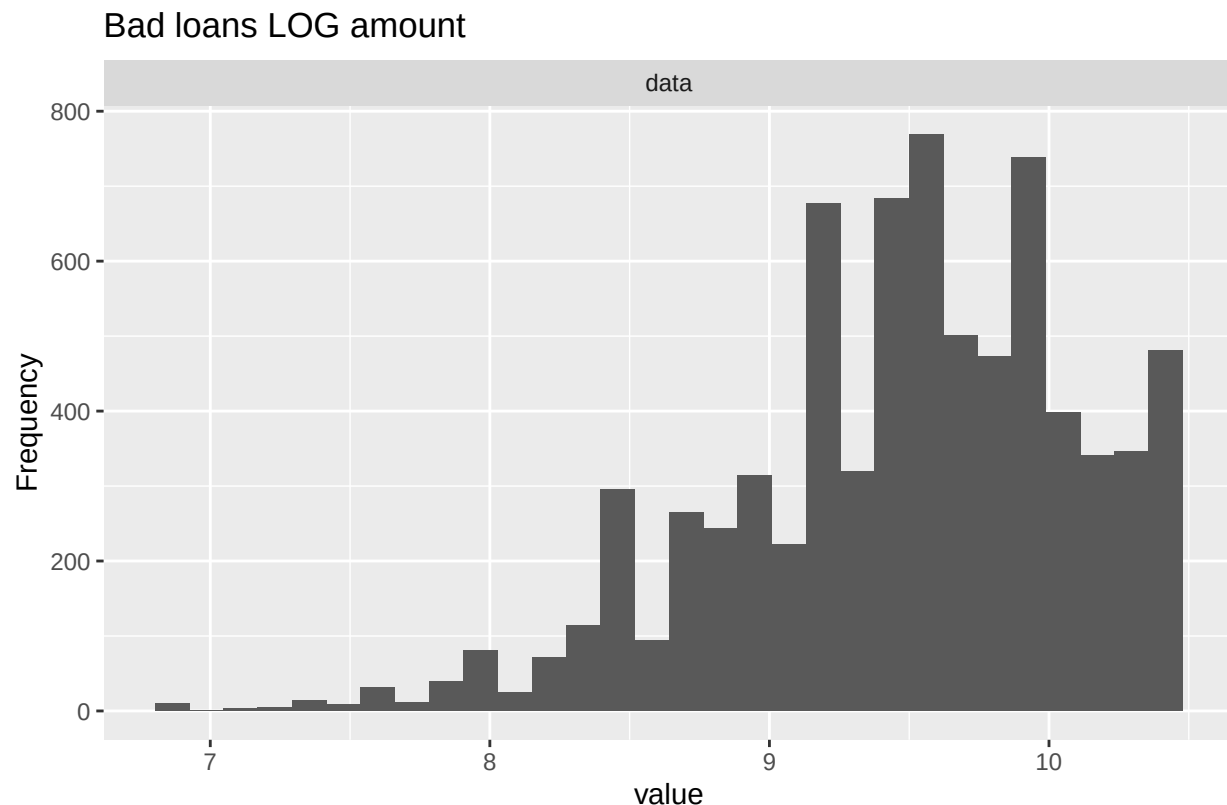
good_loan_log <- log1p(good_loan_numeric)
bad_loan_log <- log1p(bad_loan_numeric)

plot_histogram(good_loan_log$totalIilLim,title="Good loans LOG TotalIilim")
```

Good loans LOG Totallim



```
plot_histogram(bad_loan_log$amount, title="Bad loans LOG amount")
```



I checked the dataframes resulting from the `log1p` function for NA's and found none. I am not showing it to conserve space.

Next I will do some density plots with some of the different logged variables too see if there is a noticeable difference between good and bad loans. I did density plots of `openacc` and `totalLim` for good and bad loans. I didn't see much difference for `openacc` or `totalLim` between good or bad loans.

I wasn't able to eliminate many fields in my data exploration but I suspect once I start building my model, that is when I will eliminate more fields.

PART 2 - Section 5 - The Logistic Model For the start of Section 5, I will create 2 datasets from the `Loan_regrsn` dataframe from Step 4. One dataset will be my training dataset and will contain 80% of the data, the other will be my test dataset and will contain 20% of the data. I end up 6931 in `Loan_test` and 27724 in `Loan_training`

```
library(csv)
```

```
## Warning: package 'csv' was built under R version 4.1.3
```

```
training_size <- 0.8
```

```
set.seed(2112)
```

```
training_rows <- sample(seq_len(nrow(Loan_regrsn)), size = floor(training_size * nrow(Lo
```

```
Loan_training <- Loan_regrsn[training_rows, ]
Loan_test <- Loan_regrsn[-training_rows, ]
```

Next I will run the full model and use the summary function to see which variables are significant and I should include in my model. Based upon the coefficients with significant p values I will keep the following values: grade,verified,reason,term,income,revolRatio,totalAcc,totalRevLim,accOpen. When I run this model the p=values for the following fields are no longer significant:bcOpen,totalRevBal and totalIllLim, so I will drop then and try a third time. A strange thing happened When I ran the third model, T p-values was significant at .013 but it was higher than my second model which had a p-value of .012. The McFadden pseudo R Squared for the third model (.09414) was less than the second model (.09462) but just barely.Looking at the model only 3 of the 12 values of the reason variable were significant so I decided to drop the reason variable and try it again. The results were much better for the 4th time. The p-value is .0006 with a McFadden Pseudo R2 of .0932. This is the model I will move forward with. To save space, I will only show the p-value for the first and second models.

```
Loantrain_one <-glm(status2~.,data=Loan_training,family="binomial")
summary(Loantrain_one)
```

```
##
## Call:
## glm(formula = status2 ~ ., family = "binomial", data = Loan_training)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.8238   0.3054   0.5147   0.7300   1.5749
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.753e+00  2.391e+00  -0.733  0.463311
## grade1        -4.428e-01  7.582e-02  -5.840  5.22e-09 ***
## grade2        -8.676e-01  9.158e-02  -9.474  < 2e-16 ***
## grade3        -1.080e+00  1.223e-01  -8.831  < 2e-16 ***
## grade4        -1.172e+00  1.571e-01  -7.456  8.93e-14 ***
## grade5        -1.359e+00  2.161e-01  -6.291  3.16e-10 ***
## grade6        -1.339e+00  2.860e-01  -4.681  2.85e-06 ***
## length1 year    3.394e-02  8.041e-02   0.422  0.672926
## length10+ years -3.187e-02  6.159e-02  -0.517  0.604877
## length2 years   2.008e-02  7.531e-02   0.267  0.789759
## length3 years   7.293e-03  7.704e-02   0.095  0.924582
## length4 years  -1.001e-01  8.238e-02  -1.216  0.224149
## length5 years   7.034e-03  8.339e-02   0.084  0.932782
```

```

## length6 years      -1.033e-01  8.920e-02  -1.158  0.247050
## length7 years      -6.277e-03  8.984e-02  -0.070  0.944302
## length8 years      -9.805e-02  8.726e-02  -1.124  0.261120
## length9 years      -8.004e-02  9.661e-02  -0.828  0.407396
## lengthn/a          -4.415e-01  8.464e-02  -5.216  1.83e-07 ***
## homeRENT           -1.697e-01  4.015e-02  -4.225  2.38e-05 ***
## verifiedVerified    -9.596e-02  3.864e-02  -2.484  0.013007 *
## reasoncredit_card   -2.382e-01  1.982e-01  -1.202  0.229364
## reasondebt_consolidation -2.090e-01  1.957e-01  -1.068  0.285604
## reasonhome_improvement -2.811e-01  2.060e-01  -1.365  0.172403
## reasonhouse         -8.131e-02  3.182e-01  -0.256  0.798331
## reasonmajor_purchase -2.935e-01  2.275e-01  -1.290  0.197106
## reasonmedical        -5.424e-01  2.388e-01  -2.271  0.023123 *
## reasonmoving         -6.118e-01  2.605e-01  -2.348  0.018872 *
## reasonother          -2.701e-01  2.065e-01  -1.308  0.190910
## reasonrenewable_energy -5.024e-01  5.351e-01  -0.939  0.347779
## reasonsmall_business -8.160e-01  2.397e-01  -3.405  0.000663 ***
## reasonvacation       -2.386e-01  2.802e-01  -0.851  0.394543
## reasonwedding        7.549e+00  7.246e+01   0.104  0.917032
## term60 months       -8.521e-01  2.550e-01  -3.341  0.000834 ***
## amount              3.480e-01  6.818e-01   0.510  0.609754
## payment             -6.274e-01  6.824e-01  -0.919  0.357949
## income              3.285e-01  4.248e-02   7.735  1.03e-14 ***
## openAcc             -2.730e-01  2.178e-01  -1.253  0.210133
## revolRatio          -4.218e-01  1.375e-01  -3.068  0.002153 **
## totalAcc            2.392e-01  4.985e-02   4.798  1.61e-06 ***
## totalBal            -4.114e-02  2.030e-01  -0.203  0.839400
## totalRevLim         1.185e-01  3.311e-02   3.579  0.000345 ***
## accOpen24           -4.309e-01  3.507e-02 -12.288 < 2e-16 ***
## avgBal              6.327e-02  1.937e-01   0.327  0.743889
## bcOpen              2.759e-02  1.030e-02   2.678  0.007407 **
## totalLim            1.385e-01  7.594e-02   1.824  0.068183 .
## totalRevBal         -7.702e-02  3.515e-02  -2.191  0.028451 *
## totalBcLim          -6.802e-03  1.559e-02  -0.436  0.662569
## totalIllLim         -1.339e-02  6.622e-03  -2.022  0.043166 *
## bcRatio             -8.130e-04  5.904e-04  -1.377  0.168522
## debtIncRat          2.952e-03  1.896e-03   1.557  0.119428
## rate               -3.855e-02  3.663e-01  -0.105  0.916206
## totalPaid           3.328e-06  2.931e-06   1.135  0.256218
## amount2            -2.301e-06  3.484e-06  -0.660  0.508945
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##

```

```
##      Null deviance: 29029  on 27723  degrees of freedom
## Residual deviance: 26034  on 27671  degrees of freedom
## AIC: 26140
##
## Number of Fisher Scoring iterations: 8
```

```
r1 <- PseudoR2(Loantrain_one)
r1[1]
```

```
## McFadden
## 0.1031769
```

```
Loantrain_two <- glm(status2~grade+ verified + reason + term + income + revolRatio +tot
summary(Loantrain_two)
```

```
##
## Call:
## glm(formula = status2 ~ grade + verified + reason + term + income +
##      revolRatio + totalAcc + totalRevLim + accOpen24 + bcOpen +
##      totalRevBal + totalIllim, family = "binomial", data = Loan_training)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.7170   0.3205   0.5252   0.7339   1.5311
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -1.042227   0.415460  -2.509 0.012121 *
## grade1         -0.497707   0.069785  -7.132 9.89e-13 ***
## grade2         -0.971415   0.069791 -13.919 < 2e-16 ***
## grade3         -1.249296   0.075204 -16.612 < 2e-16 ***
## grade4         -1.395780   0.083612 -16.693 < 2e-16 ***
## grade5         -1.643300   0.106451 -15.437 < 2e-16 ***
## grade6         -1.730714   0.175744  -9.848 < 2e-16 ***
## verifiedVerified -0.210499   0.037202  -5.658 1.53e-08 ***
## reasoncredit_card -0.329468   0.195830  -1.682 0.092487 .
## reasondebt_consolidation -0.284814   0.193373  -1.473 0.140784
## reasonhome_improvement -0.225720   0.203272  -1.110 0.266812
## reasonhouse     -0.160707   0.316884  -0.507 0.612050
## reasonmajor_purchase -0.326817   0.225129  -1.452 0.146588
## reasonmedical   -0.464308   0.236361  -1.964 0.049483 *
## reasonmoving    -0.595877   0.257785  -2.312 0.020804 *
```

```
## reasonother          -0.247703    0.204210   -1.213  0.225136
## reasonrenewable_energy -0.553701    0.526842   -1.051  0.293267
## reasonsmall_business  -0.826452    0.237145   -3.485  0.000492 ***
## reasonvacation        -0.136203    0.276777   -0.492  0.622647
## reasonwedding         7.403876   72.463129    0.102  0.918618
## term60 months        -0.650692    0.038132  -17.064 < 2e-16 ***
## income                0.374065    0.036415   10.272 < 2e-16 ***
## revolRatio            -0.464752    0.128700   -3.611  0.000305 ***
## totalAcc              0.146257    0.040974    3.570  0.000358 ***
## totalRevLim           0.050972    0.025287    2.016  0.043827 *
## accOpen24            -0.451888    0.032987  -13.699 < 2e-16 ***
## bcOpen                0.017414    0.010056    1.732  0.083320 .
## totalRevBal          -0.032334    0.028163   -1.148  0.250934
## totalIllLim          -0.011167    0.006426   -1.738  0.082248 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 29029  on 27723  degrees of freedom
## Residual deviance: 26282  on 27695  degrees of freedom
## AIC: 26340
##
## Number of Fisher Scoring iterations: 8
```

```
r2 <- PseudoR2(Loantrain_two)
r2[1]
```

```
## McFadden
## 0.0946262
```

```
Loantrain_three <- glm(status2~grade+ verified + reason + term + income + revolRatio +
summary(Loantrain_three)
```

```
##
## Call:
## glm(formula = status2 ~ grade + verified + reason + term + income +
##      revolRatio + totalAcc + totalRevLim + accOpen24, family = "binomial",
##      data = Loan_training)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.7009   0.3208   0.5260   0.7337   1.5221
```

```
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.01291    0.41188  -2.459 0.013924 *
## grade1        -0.49536    0.06968  -7.110 1.16e-12 ***
## grade2        -0.97236    0.06951 -13.990 < 2e-16 ***
## grade3        -1.25334    0.07478 -16.760 < 2e-16 ***
## grade4        -1.39934    0.08307 -16.845 < 2e-16 ***
## grade5        -1.64692    0.10577 -15.570 < 2e-16 ***
## grade6        -1.74163    0.17503  -9.950 < 2e-16 ***
## verifiedVerified -0.20631    0.03716  -5.552 2.83e-08 ***
## reasoncredit_card -0.33830    0.19587  -1.727 0.084137 .
## reasondebt_consolidation -0.29687    0.19341  -1.535 0.124797
## reasonhome_improvement -0.23341    0.20331  -1.148 0.250952
## reasonhouse    -0.15784    0.31684  -0.498 0.618354
## reasonmajor_purchase -0.33796    0.22512  -1.501 0.133297
## reasonmedical  -0.48817    0.23633  -2.066 0.038865 *
## reasonmoving   -0.61920    0.25773  -2.403 0.016283 *
## reasonother    -0.25935    0.20423  -1.270 0.204122
## reasonrenewable_energy -0.55135    0.52666  -1.047 0.295156
## reasonsmall_business -0.82574    0.23700  -3.484 0.000494 ***
## reasonvacation -0.13705    0.27685  -0.495 0.620574
## reasonwedding   7.40781   72.46313   0.102 0.918575
## term60 months  -0.65411    0.03797 -17.228 < 2e-16 ***
## income          0.34619    0.03474   9.964 < 2e-16 ***
## revolRatio     -0.62515    0.10952  -5.708 1.14e-08 ***
## totalAcc        0.10677    0.03872   2.758 0.005824 **
## totalRevLim     0.07121    0.02186   3.257 0.001124 **
## accOpen24       -0.46665    0.03259 -14.320 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 29029  on 27723  degrees of freedom
## Residual deviance: 26296  on 27698  degrees of freedom
## AIC: 26348
##
## Number of Fisher Scoring iterations: 8

r3 <- PseudoR2(Loantrain_three)
r3[1]

## McFadden
## 0.0941462
```

```
Loantrain_Four <- glm(status2~grade + verified + term + income + revolRatio +totalAcc
```

```
summary(Loantrain_Four)
```

```
##
## Call:
## glm(formula = status2 ~ grade + verified + term + income + revolRatio +
##      totalAcc + totalRevLim + accOpen24, family = "binomial",
##      data = Loan_training)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.6806   0.3228   0.5263   0.7347   1.4497
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.25375    0.36781  -3.409 0.000653 ***
## grade1        -0.49820    0.06946  -7.172 7.37e-13 ***
## grade2        -0.97831    0.06865 -14.250 < 2e-16 ***
## grade3        -1.26149    0.07319 -17.235 < 2e-16 ***
## grade4        -1.41071    0.08100 -17.417 < 2e-16 ***
## grade5        -1.66973    0.10347 -16.137 < 2e-16 ***
## grade6        -1.76051    0.17298 -10.177 < 2e-16 ***
## verifiedVerified -0.20589    0.03710  -5.550 2.86e-08 ***
## term60 months  -0.64593    0.03737 -17.286 < 2e-16 ***
## income         0.34222    0.03426   9.990 < 2e-16 ***
## revolRatio     -0.62001    0.10649  -5.822 5.81e-09 ***
## totalAcc       0.10785    0.03864   2.791 0.005248 **
## totalRevLim    0.06733    0.02164   3.112 0.001859 **
## accOpen24      -0.45874    0.03239 -14.165 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 29029  on 27723  degrees of freedom
## Residual deviance: 26322  on 27710  degrees of freedom
## AIC: 26350
##
## Number of Fisher Scoring iterations: 5
```

```
r4 <- PseudoR2(Loantrain_Four)
r4[1]
```

```
## McFadden
## 0.09326458
```

Below I will use my model created above to predict the status for loans in the test data. After I use my model to predict the status of the loans, I will create a confusion matrix to determine the overall accuracy of the model. The results of the confusion matrix for the model are as follows: I have 5260 loans correctly predicted as good loans and 154 loans correctly predicted as bad loans. So I have a total accuracy percentage of 78%. 98 percent of good loans were correctly predicted as good while only 11 percent of bad loans were correctly predicted as bad. While this model leaves a bit to be desired in predicting the bad loans accurately, it does a much better job accurately predicting the good ones.

```
test_model <- glm(status2~grade + verified + term + income + revolRatio +totalAcc + to
probabilities <- predict(test_model,newdata=Loan_test, type="response")
```

```
t <- .5
```

```
Bad_or_Good <- ifelse(probabilities>t,1,0)
Bad_or_Good <- as.factor(Bad_or_Good)
```

```
cf <- confusionMatrix(data=Bad_or_Good,reference=Loan_test$status2)
cf$table
```

```
##           Reference
## Prediction    0    1
##           0  126 106
##           1 1429 5270
```

```
cf$overall[1]
```

```
## Accuracy
## 0.7785312
```

## Section 6 - Optimizing the Threshold for Accuracy.

In this section I am going to vary the threshold from .5 to attempt to correctly predict more bad loans. I will create a confusion matrix for each different threshold and then I will graph them all to show accuracy vs. threshold. As you recall from the previous section my accuracy percentage was 78% but was a much better predictor of 'good' loans than 'bad'

loans. I will calculate the accuracy for 5 different thresholds between 0 and 1 to attempt to predict a better percentage of bad loans. Due to space constraints, I will just show the confusion matrix containing the predictions and the accuracy value for each calculation.

```
# 6 different thresholds
```

```
th1 <- .30  
th2 <- .45  
th3 <- .55  
th4 <- .60  
th5 <- .75
```

```
Bad_or_Good1 <- ifelse(probabilities>th1,1,0)  
Bad_or_Good1 <- as.factor(Bad_or_Good1)  
c1 <- confusionMatrix(data=Bad_or_Good1,reference=Loan_test$status2)
```

```
## Warning in confusionMatrix.default(data = Bad_or_Good1, reference =  
## Loan_test$status2): Levels are not in the same order for reference and data.  
## Refactoring data to match.
```

```
c1$table
```

```
##           Reference  
## Prediction    0    1  
##           0    0    0  
##           1 1555 5376
```

```
c1$overall[1]
```

```
## Accuracy  
## 0.7756456
```

```
Bad_or_Good2 <- ifelse(probabilities>th2,1,0)  
Bad_or_Good2 <- as.factor(Bad_or_Good2)  
c2 <- confusionMatrix(data=Bad_or_Good2,reference=Loan_test$status2)  
c2$table
```

```
##           Reference  
## Prediction    0    1  
##           0    55   35  
##           1 1500 5341
```

```
c2$overall[1]
```

```
## Accuracy  
## 0.7785312
```

```
Bad_or_Good3 <- ifelse(probabilities>th3,1,0)  
Bad_or_Good3 <- as.factor(Bad_or_Good3)  
c3 <- confusionMatrix(data=Bad_or_Good3,reference=Loan_test$status2)  
c3$table
```

```
##           Reference  
## Prediction    0    1  
##           0  215  204  
##           1 1340 5172
```

```
c3$overall[1]
```

```
## Accuracy  
## 0.7772327
```

```
Bad_or_Good4 <- ifelse(probabilities>th4,1,0)  
Bad_or_Good4 <- as.factor(Bad_or_Good4)  
c4 <- confusionMatrix(data=Bad_or_Good4,reference=Loan_test$status2)  
c4$table
```

```
##           Reference  
## Prediction    0    1  
##           0  349  381  
##           1 1206 4995
```

```
c4$overall[1]
```

```
## Accuracy  
## 0.7710287
```

```
Bad_or_Good5 <- ifelse(probabilities>th5,1,0)  
Bad_or_Good5 <- as.factor(Bad_or_Good5)  
c5 <- confusionMatrix(data=Bad_or_Good5,reference=Loan_test$status2)  
c5$table
```

```
##           Reference
## Prediction    0    1
##           0  908 1513
##           1  647 3863
```

```
c5$overall[1]
```

```
## Accuracy
## 0.6883567
```

Given the data in the confusion matrices above, I the accuracy vs the threshold of each of them - including the default (.5). So I did a confusion matrix for 5 thresholds in addition to the default. The values are: .30, .45, .55, .60, .75, .85. The highest accuracy level I get is at the .45 and the default threshold. My accuracy percentages are 78.1% for both the default threshold and 78.09 for the .12 threshold. The .45 threshold does a little better job at predicting 'good' loans with 5,341 correctly predicted as 'good' compared to 5,260 for the default threshold, but the .5 threshold does a better job at predicting 'bad' loans with 116 'bad' loans predicted correctly. That number is 35 for the .45 threshold. The lower the threshold is, the better it is at predicting good loans. Unfortunately, the lower the threshold, the worse it is at predicting 'bad' loans. The lowest threshold I used, .3, is 77.5% accurate with 5376 out of 5377 loans correctly predicted as good but it incorrectly predicts all 'bad' loans as good. As the threshold increases from the default level, the overall accuracy drops as the number of 'good' loans predicted correctly drops but the number of 'bad' loans predicted correctly increases. For the .55 threshold, the overall accuracy drops slightly to 77.3%. The number of correctly predicted 'good' loans drops to 4907 but the number of 'bad' loans rises to 469. As the threshold values rise, the number of predicted 'good' loans drops and the number of predicted 'bad' loans rises. With the threshold set at .65 the number of 'good' loans is 4907 while the number of bad loans is 409. From the .65 threshold to the .75 threshold the numbers take a dramatic jump. At the .75 threshold there are 3729 loans that are 'good' and 958 'bad' loans. This leaves us with over 2,244 loans that are incorrectly classified as good or bad. The reverse is true as well since as we decrease the threshold from .5 the number of loans classified as good rises and the number of loans classified as bad drops. The accuracy percentage drops as well.

```
## Loading required package: plyr
```

```
## -----
```

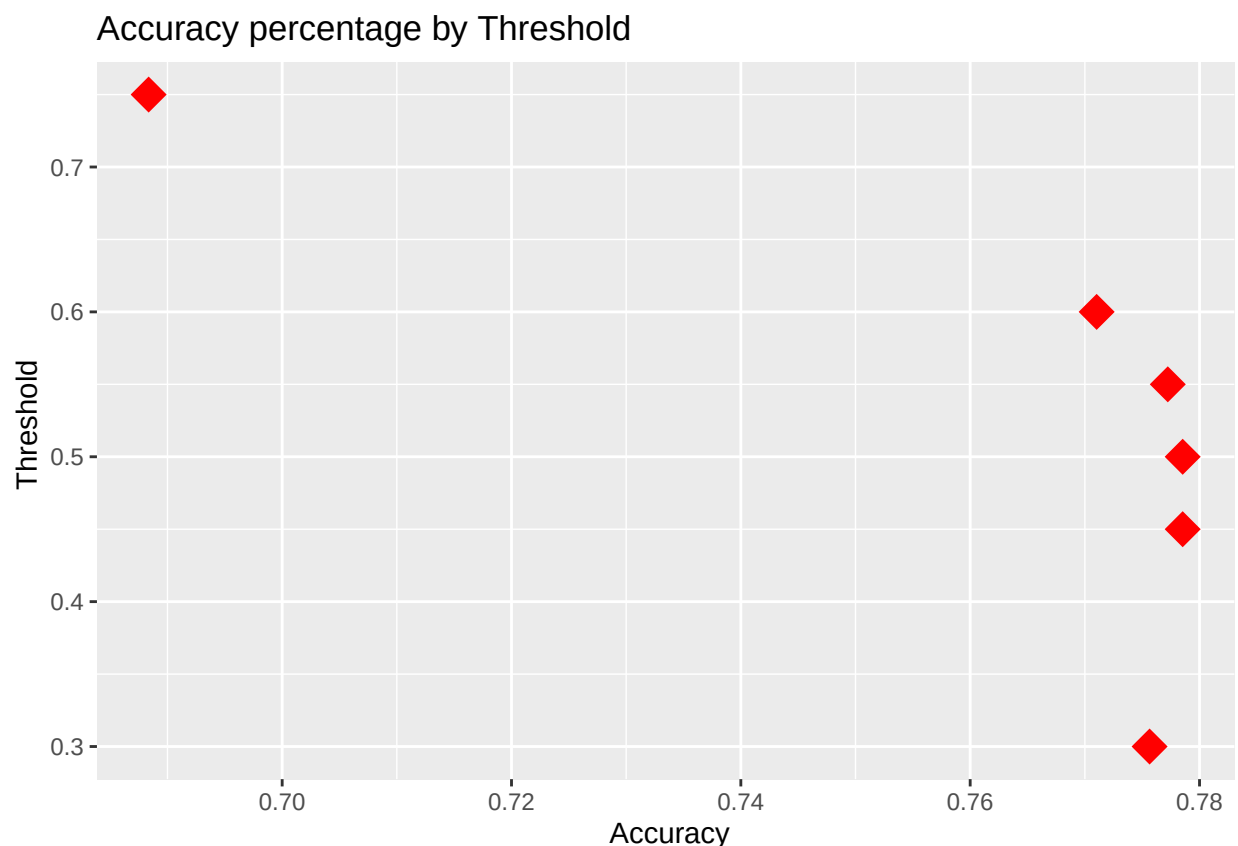
```
## You have loaded plyr after dplyr - this is likely to cause problems.
## If you need functions from both plyr and dplyr, please load plyr first, then dplyr:
## library(plyr); library(dplyr)
```

```
## -----
```

```
##
## Attaching package: 'plyr'

## The following objects are masked from 'package:dplyr':
##
##   arrange, count, desc, failwith, id, mutate, rename, summarise,
##   summarize

## The following object is masked from 'package:purrr':
##
##   compact
```



Part 7 - Optimizing the Threshold for Profit In this section I will take the predictions at the different threshold levels from Part 6 and apply them to my test data. Then I will sum the profit for each threshold and determine what the maximum profit increase is for the loans I predicted as good. I add the probabilities calculations from section 5 and the test data I created in section 5. Then I sum the total paid minus the amount for my total profit. According to the model, the lowest threshold of .3 is where the biggest profit is - 1,418,710. For there the profit for the .45 threshold is 1,374,321. The default threshold profit (.5) is 1,386,706. The profit for .55 is 1,354,828. The profit keeps decreasing from there. At .65 the profit is 98,274.20 then for .75 threshold the profit is 966,017.50. The total profit for all

the loans in the training set is 1,418,710. The profit for a ‘perfect’ model that denies all of the truly bad loans is 750778.40.

```
Loan_test_th1 <- cbind(Loan_test,probabilities)

thr1 <- sqldf(c("select sum(totalPaid - amount2) from Loan_test_th1 where probabilities
thr2<-  sqldf(c("select sum(totalPaid - amount2)  from Loan_test_th1 where probabilities
thrdf  <- sqldf(c("select sum(totalPaid - amount2)   from Loan_test_th1 where probabilit
thr3<-  sqldf(c("select sum(totalPaid - amount2)  from Loan_test_th1 where probabilities
thr4<-  sqldf(c("select sum(totalPaid - amount2)  from Loan_test_th1 where probabilities
thr5<-  sqldf(c("select sum(totalPaid - amount2)  from Loan_test_th1 where probabilities
Prof_no_model <- sqldf(c("select sum(totalPaid - amount2)  from Loan_test_th1 where stat
tot_profit <- sqldf(c("select sum(totalPaid - amount2)  from Loan_test_th1"))

thr1

##  sum(totalPaid - amount2)
## 1                1418710

thr2

##  sum(totalPaid - amount2)
## 1                1374321

thrdf

##  sum(totalPaid - amount2)
## 1                1386706

thr3

##  sum(totalPaid - amount2)
## 1                1354828
```

thr4

```
## sum(totalPaid - amount2)
## 1 982747.2
```

thr5

```
## sum(totalPaid - amount2)
## 1 966017.5
```

Prof\_no\_model

```
## sum(totalPaid - amount2)
## 1 750778.4
```

tot\_profit

```
## sum(totalPaid - amount2)
## 1 1418710
```